Repaso examen Tema 2 – Instalación de Sistemas Operativos

Nov 10, 2026 · @Fran

Repaso del Tema 2 con lo que el profesor ha dicho que entra seguro: tipos de arquitectura y virtualización (1), procesos y memoria (2), licencias (3), sistemas de archivos (4), particionado con sus limitaciones y el ejercicio (5), cómo actualizar el SO (6), hipervisores (7), contenedores (8), Docker (9) y WSL por encima (11). Las buenas prácticas de Docker (10) no entran. Examen: viernes 13 de noviembre.

Apartado 1 · Tipos de arquitectura y virtualización

Entra: los cuatro tipos de arquitectura y qué es la virtualización.

Tipos de arquitectura

	Monolítica	Por capas	Microkernel	Híbrida
Idea	Todo el SO en un único espacio de direcciones, en modo kernel	Niveles jerárquicos: cada capa usa la de abajo y sirve a la de arriba	Solo lo esencial (procesos, memoria, IPC) en modo kernel; archivos y controladores en modo usuario	Mezcla: unos servicios integrados en el kernel y otros en modo usuario
Rendimiento	Alto (sin coste de comunicación entre módulos)	Medio (llamadas entre capas)	Medio-bajo (comunicación entre procesos)	Alto en los servicios integrados
Modularidad	Baja: un cambio obliga a recompilar todo	Alta: cada capa se modifica por separado	Muy alta: servicios añadidos o quitados en caliente	Media-alta
Seguridad	Baja: un fallo tumba todo	Media	Alta: un fallo en un servicio no afecta al kernel	Media-alta
Ventaja principal	Rendimiento y sencillez	Modularidad y mantenimiento	Estabilidad y seguridad	Equilibrio rendimiento/modularidad
Inconveniente	Difícil de mantener, poco robusta	Sobrecarga entre capas	Sobrecarga de comunicación	Diseño complejo

	Monolítica	Por capas	Microkernel	Híbrida
Ejemplos	MS-DOS, primeros Unix, núcleo de Linux	Windows NT, THE OS	QNX, MINIX, L4	Windows desde NT, macOS, Linux con módulos cargables

Para recordarlo: monolítica = todo junto y rápido pero frágil; microkernel = lo mínimo en el kernel, seguro pero más lento; por capas = pisos; híbrida = un poco de cada.

Qué es la virtualización

Virtualizar es ejecutar **varios sistemas operativos independientes en un mismo equipo**, cada uno en una **máquina virtual (VM)** que gestiona un **hipervisor** (apartado 7).

- **Ventajas:** aprovechar mejor los recursos, aislamiento entre sistemas, menor coste y facilidad para probar entornos.
- **Tipo 1 (bare-metal):** sobre el hardware; servidores (VMware ESXi, Hyper-V, Proxmox).
- **Tipo 2 (alojado):** como aplicación sobre un SO; equipos personales (VirtualBox, VMware Workstation).
- Los **contenedores** son otra forma de virtualizar, a nivel de sistema operativo: comparten el kernel del anfitrión (apartado 8).

Apartado 2 · Elementos de un sistema operativo

Entran la gestión de procesos (1.1) y la gestión de memoria (1.2).

1.1 Gestión de procesos

Un **proceso** es un programa en ejecución que necesita recursos (CPU, memoria, archivos). El SO crea, suspende, reanuda y termina procesos, asigna la CPU (planificación) y gestiona la comunicación entre procesos (IPC).

Estados: Nuevo → Listo (admisión) → Ejecutando (planificador) → Bloqueado (espera de E/S) → Listo al acabar la E/S → Terminado.

- *Nuevo*: creado, aún no cargado en memoria. *Listo*: en memoria, esperando CPU. *Ejecutando*: usando la CPU. *Bloqueado*: espera un evento externo (disco, entrada del usuario). *Terminado*: ha finalizado.

Algoritmo de planificación	Idea	Ventaja	Desventaja
FCFS	Por orden de llegada	Simple, justo	Efecto convoy

Algoritmo de planificación	Idea	Ventaja	Desventaja
SJF	El más corto primero	Menor espera media	Difícil predecir la duración
Round Robin	Cuantos de tiempo cíclicos	Evita monopolio de CPU	Puede aumentar la espera
Prioridad	Mayor prioridad primero	Flexible	Riesgo de inanición
Multinivel	Combina algoritmos por tipo de proceso	Adaptable	Compleja de implementar

IPC:

1. Memoria compartida (p. ej. POSIX Shared Memory).
2. Paso de mensajes: pipes (unidireccionales, como `ls | grep "txt"`) y colas de mensajes (bidireccionales).
3. Semáforos: sincronizan y evitan condiciones de carrera.
4. Sockets: procesos en distintas máquinas (TCP/IP).

1.2 Gestión de memoria

Jerarquía (de más rápida y cara a más lenta y barata): registros ~1 ns → L1 ~1-4 ns → L2/L3 ~10-50 ns → RAM ~100 ns → SSD ~10-100 µs → HDD ~5-10 ms → cinta, segundos.

Técnica	Idea	Ventaja	Desventaja
Partición fija	Particiones estáticas	Simple	Fragmentación interna
Partición dinámica	Según el tamaño del proceso	Sin fragmentación interna	Fragmentación externa
Paginación	Marcos físicos y páginas lógicas	Sin fragmentación externa	Sobrecarga de tablas
Segmentación	Segmentos lógicos (código, datos, pila)	Protección y compartición	Fragmentación externa
Memoria virtual	Paginación + disco para simular más RAM	Ejecutar programas grandes	Coste de acceso a disco

Paginación: página = bloque lógico (p. ej. 4 KB) · marco = bloque físico · tabla de páginas = el mapeo, con bits de presencia, modificación y referencia · *page fault* = la página no está en RAM y se carga desde disco o swap. En el ejemplo del temario, la página 0 está en el marco 5, la 1 no está presente y la 2 está en el marco 10.

Apartado 3 · Licencias

Los sistemas se comparan por requisitos, características, campo de aplicación y licencia (criterio RA2.c).

1.1 Tipos de software según la licencia

Tipo	Qué permite	Ejemplos
Software libre	Usar para cualquier fin, estudiar el código, modificarlo y redistribuirlo	Linux, FreeBSD, LibreOffice
Código abierto	Código accesible y licencia que permite usarlo y modificarlo; en la práctica se solapa con el libre	Base de Android, Chromium
Propietario	El fabricante controla el código; el usuario solo tiene derecho de uso según el contrato (EULA)	Windows, macOS
Freeware	Gratuito pero con código cerrado	Algunos controladores y utilidades
Shareware / prueba	Gratis un tiempo o con funciones limitadas; después se paga	—

Dentro del libre: **copyleft** (GPL: las obras derivadas mantienen la misma libertad; el núcleo Linux usa GPL) y **permisivas** (MIT, BSD: permiten meter el código en productos cerrados).

1.2 Licencias de Windows

- **OEM:** preinstalada en un equipo nuevo (o comprada para montarlo); queda ligada a ese hardware.
- **Retail:** se compra suelta y, en general, se puede pasar a otro equipo.
- **Por volumen:** empresas y centros educativos, con una clave o contrato para muchos equipos.
- **Windows Server:** reglas propias, por núcleos de procesador y licencias de acceso de cliente.

- Antes de desplegar un sistema propietario hay que comprobar que la licencia cubre el número de equipos y el uso. Hay versiones de evaluación gratuitas y temporales para practicar.

1.3 Tabla comparativa

Sistema	Licencia	Campo habitual	Observación
Windows 10/11	Propietaria	Escritorio, ofimática, juegos	Requisitos mínimos según versión
Windows Server	Propietaria	Servidores de empresa	—
Ubuntu, Debian, Fedora…	Libre (GPL y otras)	Escritorio, servidores, desarrollo	Varias distribuciones con un mismo núcleo
macOS	Propietaria	Escritorio de Apple	Solo en hardware de Apple
Android	Libre en la base, con componentes propietarios	Móviles y tabletas	Basado en el núcleo Linux

Apartado 4 · Sistemas de archivos

Entra todo.

1. Agrupación de registros

- **Registro:** unidad lógica de acceso a la información. **Bloque:** unidad física mínima de transferencia en disco.
- Los registros se agrupan en bloques de tamaño fijo, de longitud variable por tramos o sin tramos.
- Asignación de memoria: predefinida (espacio fijo al crear el archivo) o dinámica (según crece).
- **Contigua:** bloques consecutivos; eficiente en acceso secuencial, pero fragmenta.
- **Encadenada:** cada bloque apunta al siguiente; no fragmenta, lectura lenta.
- **Indexada:** un índice guarda las direcciones de los bloques; no fragmenta, acceso secuencial y aleatorio; la más usada.
- Las tablas de asignación registran el espacio libre y ocupado (de bits o listas).

2. FAT32

Evolución de FAT16 y FAT12 (Microsoft), con tabla de asignación de 32 bits. Archivo máx. **4 GB**; partición máx. **32 GB** en la práctica (2 TB en teoría). Compatible con Windows, macOS (solo lectura), Linux, consolas y embebidos. Sin permisos ni journaling; fragmentación alta; nombres de 255 caracteres Unicode. Ventajas: compatibilidad universal, pocos recursos, sencillez. Desventajas: límite de 4 GB, sin seguridad multiusuario, fragmentación, más riesgo de corrupción. Uso: USB, SD, discos para compartir entre sistemas, dispositivos antiguos o embebidos.

3. NTFS

Sistema principal de Microsoft (desde Windows NT, usado desde Windows 2000). Usa la **MFT** (Master File Table) con los metadatos de todos los archivos. Archivo y partición hasta **16 EB** en teoría. Windows completo; Linux lectura/escritura con controladores; macOS solo lectura. Permisos **ACL**, journaling, compresión nativa, cifrado **EFS**, cuotas, enlaces duros y simbólicos, nombres de 255 caracteres. Ventajas: rendimiento en discos grandes, seguridad (permisos, cifrado, auditoría), recuperación ante fallos. Desventajas: menos compatible fuera de Windows, más recursos. Uso: discos internos de Windows, Windows Server.

4. ext4

El más usado en Linux; evolución de ext3 y ext2 (2008). Basado en **inodos**. Archivo máx. **16 TB** (bloques de 4 KB); partición **1 EB** en teoría. Linux completo, Android y algunos embebidos. Permisos Unix (usuario, grupo, otros), journaling (completo, ordenado o solo metadatos), enlaces, nombres de 255 caracteres UTF-8, **extents**. Sin compresión nativa (e2compr) ni cifrado nativo (eCryptfs o LUKS). Ventajas: rendimiento, escalabilidad, recuperación rápida, permisos, buen comportamiento con SSD. Desventajas: Windows no lo lee de forma nativa; sin compresión nativa. Uso: discos y SSD de Linux, servidores, Raspberry Pi.

5. Tabla comparativa

Característica	FAT32	NTFS	ext4
Sistema principal	Todos	Windows	Linux
Archivo máx.	4 GB	16 EB	16 TB
Partición máx.	32 GB (práctico)	16 EB	1 EB
Journaling	No	Sí	Sí

Característica	FAT32	NTFS	ext4
Permisos	No	Sí (ACL)	Sí (Unix)
Windows	Sí	Sí	No (sin controladores)
Linux	Sí	Sí (lectura/escritura)	Sí
macOS	Solo lectura	Solo lectura	No
Fragmentación	Alta	Baja	Baja
Compresión nativa	No	Sí	No
Cifrado nativo	No	Sí (EFS)	No (externo)
Enlaces simbólicos	No	Sí	Sí
Uso típico	USB, SD	Discos de Windows	Discos de Linux

6. ¿Cuál elegir?

- **FAT32:** compatibilidad universal, sin archivos de más de 4 GB y sin permisos.
- **NTFS:** usas Windows y necesitas seguridad (permisos, cifrado), archivos grandes o journaling.
- **ext4:** usas Linux y quieres rendimiento en HDD o SSD, o particiones y archivos muy grandes.

7. Otros sistemas de archivos

- **exFAT** (Microsoft): sin límite de 4 GB por archivo; Windows, macOS y Linux (con controladores); ideal para USB y SD grandes; sin permisos ni journaling.
- **Btrfs** (Linux): journaling avanzado, snapshots, compresión transparente, deduplicación, RAID por software; menos maduro que ext4.
- **ZFS** (Sun, ahora Oracle): integridad con checksums, snapshots, clonación, compresión, pools y RAID avanzado; mucha RAM (al menos 4 GB); no es nativo en Windows.

Apartado 5 · Particionado de discos

Entra todo, y además las limitaciones de cada esquema con su porqué y el ejercicio de particionado (puntos 7 y 8).

1. Concepto

Particionar es dividir un disco (HDD/SSD) en secciones lógicas independientes. Cada partición puede tener su propio sistema de archivos, alojar datos o un SO y gestionarse como unidad aparte (C:, D:, /home, /boot). Ventajas: organizar datos, varios sistemas en un disco, mejor rendimiento, copias y recuperación más fáciles.

2. Tipos de particiones

- **Primarias:** contienen directamente un SO o datos. Límite: **4 en MBR**, **128 en GPT**. C: suele ser primaria.
- **Extendida:** contenedor de lógicas. **Solo una por disco y solo en MBR**; ocupa una de las 4 ranuras primarias.
- **Lógicas:** dentro de la extendida; en teoría ilimitadas (según el espacio). En Linux se numeran **desde 5** (`/dev/sda5`). Necesitan un gestor como GRUB para arrancar.

3. Esquemas: MBR y GPT

	MBR	GPT
Tabla	Primeros 512 bytes del disco	GUIDs; copia de la tabla al final del disco
Tamaño de disco	Hasta 2 TB	Hasta 8 ZiB
Particiones	4 primarias (o 3 primarias + 1 extendida)	Hasta 128 primarias, sin extendidas ni lógicas
Firmware	BIOS (legacy)	Requiere UEFI
Uso	Sistemas antiguos	Sistemas modernos

4. Particionado típico en Windows

- **MBR (BIOS):** C: (primaria NTFS), D: (primaria NTFS) y una extendida con una lógica NTFS de recuperación. Windows suele añadir una partición oculta de recuperación de 100-500 MB.
- **GPT (UEFI):** partición EFI (FAT32), C: (NTFS), D: (NTFS) y **MSR de 16 MB** reservada.

5. Particionado típico en Linux

- **MBR (BIOS):** /boot (primaria ext4) y una extendida con / (lógica ext4), /home (lógica ext4) y swap (lógica).

- **GPT (UEFI):** /boot/efi (FAT32, ESP obligatoria), /boot, /, /home (ext4) y swap, todas primarias.
- /boot suele ser primaria en MBR para evitar problemas con la BIOS. La swap es opcional y se recomienda si la RAM es insuficiente.

6. Herramientas

- **Windows:** `diskpart` (CLI) y Administrador de discos (GUI).
- **Linux:** `fdisk`, `gdisk`, `parted` (CLI); GParted y KDE Partition Manager (GUI).

7. Limitaciones de MBR y GPT, y su porqué

Limitación de MBR	Por qué pasa	Cómo se resuelve
Solo **4 particiones primarias**	La tabla está en los **primeros 512 bytes** del disco y solo tiene sitio para 4 entradas	Convertir una en **extendida** y crear dentro **lógicas** (en Linux desde `sda5`); o usar GPT (128 primarias)
Discos de **hasta 2 TB** (2 TiB)	Guarda el inicio y el tamaño de cada partición en campos de **32 bits**, contados en sectores de **512 bytes**: $2^{32} \times 512$ B = 2 TiB	Lo que pase de 2 TiB queda sin usar; con discos más grandes, GPT (hasta 8 ZiB)
Solo **un gestor de arranque**	Con BIOS, el código de arranque va en el MBR: el último sistema que se instala sobrescribe al anterior	Con UEFI cada sistema deja su `.efi` en la partición EFI y conviven
Una sola copia de la tabla	Si se daña el principio del disco, se pierden las particiones	GPT guarda una **copia de la tabla al final** del disco
Solo **una extendida** por disco	Ocupa una de las 4 ranuras	—

Limitaciones de GPT: necesita **UEFI**; Windows solo arranca desde GPT con UEFI. Con BIOS, Linux sí puede arrancar desde GPT si GRUB tiene una pequeña partición **BIOS boot** de 1 MB. Algunas BIOS muy antiguas dan problemas con GPT.

Regla para elegir: UEFI → GPT. BIOS antigua o Windows en equipo sin UEFI → MBR. Disco de más de 2 TB o más de 4 particiones sin líos → GPT.

8. Cómo resolver un ejercicio de particionado

1. **Firmware:** UEFI → GPT; BIOS → MBR (o GPT solo para Linux, con BIOS boot).

2. **Cuenta las particiones:** con MBR y más de 4, haz 3 primarias + 1 extendida con las lógicas.
3. **Añade las obligatorias:** con UEFI, la partición **EFI** en FAT32 (una sola para todos los sistemas); si hay Windows en GPT, la **MSR** de 16 MB.
4. **Elige sistema de archivos:** Windows en NTFS; Linux en ext4; swap sin sistema de archivos; EFI en FAT32.
5. **Tamaños:** swap según la RAM y si hay hibernación; `/home` separado para no perder datos al reinstalar; el resto del disco para los datos.
6. **Haz la tabla:** uso, sistema de archivos, tamaño, punto de montaje o letra y nombre en Linux (`/dev/sdaN` o `/dev/nvme0n1pN`). Comprueba que **suma el disco**.
7. **Si hay dual boot:** primero Windows y después Linux.

Ejemplo A (UEFI, NVMe de 512 GB, 16 GB de RAM, Windows 11 + Ubuntu): GPT · EFI 512 MB FAT32 · MSR 16 MB · Windows 160 GB NTFS (C:) · Recuperación 1 GB · `/` 60 GB ext4 · swap 4 GB · `/home` el resto ext4; de `nvme0n1p1` a `nvme0n1p7` .

Ejemplo B (BIOS, disco de 2 TB, 6 particiones): MBR · `sda1` sistema (primaria activa) · `sda2` swap · `sda3` datos · `sda4` extendida · `sda5` - `sda7` pruebas (lógicas). El disco cabe entero (2 TB ≈ 1,82 TiB < 2 TiB).

Los dos resueltos con detalle están en el apartado 12, Tarea 2.

Apartado 6 · Cómo actualizar el sistema operativo

Entra: saber cómo se actualiza un SO.

- **Por qué:** un SO recién instalado no está listo; hay que actualizarlo.
- **Tipos de actualizaciones:** de **seguridad** (las más urgentes), de calidad o errores, y de funcionalidades (pueden necesitar copia de seguridad antes).
- **Windows:** con **Windows Update**; puede pedir reiniciar.
- **Ubuntu / Debian:** con `apt` , en dos pasos y en este orden:

```
sudo apt update && sudo apt upgrade -y
```

Parte	Qué hace
`sudo`	Ejecuta como administrador
`apt update`	Descarga la **lista** de paquetes de los repositorios
`apt upgrade`	**Instala** las versiones nuevas de los paquetes que ya tienes

Parte	Qué hace
`&&`	Ejecuta la segunda orden solo si la primera ha ido bien
`-y`	Responde «sí» a la confirmación

- **El orden importa:** sin `update` antes, `upgrade` instala según un catálogo desactualizado.
- **Después:** `sudo apt autoremove` borra paquetes que ya no hacen falta (por ejemplo, un núcleo antiguo) y, si se ha actualizado el núcleo, se reinicia con `sudo reboot`.

Apartado 7 · Hipervisores

Entra: hipervisor tipo 1 y 2; tipos de red y ecosistema, por encima.

- **Máquina virtual (VM):** ordenador simulado por software con CPU, RAM, disco y red virtuales. Su SO es el **invitado** (guest); el del equipo real, el **anfitrión** (host).
- **Hipervisor** (o monitor de máquina virtual, VMM): la capa entre el hardware y las VM. Da a cada VM hardware virtual y reparte los recursos físicos.

	Tipo 1 (bare-metal)	Tipo 2 (alojado)
Dónde va	Directamente sobre el hardware	Como un programa sobre un SO
Para qué	Servidores y centros de datos	Desarrollo, pruebas, aprender
Rendimiento	Más rendimiento y seguridad	Algo menos
Ejemplos	VMware ESXi, Hyper-V, Proxmox	VirtualBox, VMware Workstation

Tipos de red en VirtualBox (por encima)

- **NAT** (por defecto): la VM sale a Internet con la IP del anfitrión; desde fuera no se ve.
- **Adaptador puente:** la VM tiene su propia IP en la red real, como un equipo más.
- **Red interna:** solo se ven las VM entre sí.
- **Solo-anfitrión:** se ven el anfitrión y las VM, sin salida a la red.

Ecosistema (nombres, por encima)

Todos de tipo 1: **VMware vSphere/ESXi** (el líder en empresas), **Microsoft Hyper-V** (de Windows Server), **KVM** (dentro del núcleo de Linux), **Xen / Citrix Hypervisor** y **Proxmox VE** (basado en Debian, con máquinas virtuales y contenedores). **VirtualBox** es tipo 2,

gratuito y de código abierto.

Apartado 8 · Contenedores

Entra: saber qué son los contenedores y los comandos básicos (los comandos están en el apartado 9).

- **Contenedor:** unidad ligera y portable que lleva una aplicación con todo lo que necesita (código, librerías, configuración). **Comparte el kernel del anfitrión**, así que no lleva un SO completo.
- **Características:** aislamiento, portabilidad, eficiencia, escalabilidad y automatización con `Dockerfile`.
- **Herramientas:** Docker, LXC, Podman.

	Contenedores	Máquinas virtuales
Sistema operativo	Comparten el del anfitrión	Cada una con su SO invitado completo
Se ejecutan sobre	Docker Engine	Un hipervisor
Arranque	Segundos	Minutos
Recursos	Pocos	Muchos, reservados
Aislamiento	A nivel de procesos (medio)	A nivel de hardware (más seguro)
Usos	Microservicios, desarrollo, nube	Varios SO, pruebas, servidores completos

Conceptos clave:

- **Puerto publicado:** `-p 8080:80` lleva el puerto 8080 del equipo al 80 del contenedor (equipo:contenedor).
- **Volumen:** guarda datos fuera del contenedor para que no se pierdan al borrarlo.
- **Red:** por defecto `bridge`; los contenedores de una misma red se encuentran por su nombre.
- **Docker Compose:** arranca varios contenedores a la vez desde un archivo YAML (`compose.yaml`). Para entornos críticos se usa Kubernetes o Docker Swarm.

Apartado 9 · Docker

Entra: saber qué es Docker y los comandos básicos.

- **Docker:** plataforma que empaqueta una aplicación con todo lo que necesita y la ejecuta en contenedores. No reserva recursos fijos: usa los que necesita en cada momento.
- **Dockerfile:** archivo con las instrucciones para construir una imagen.
- **Imagen:** plantilla de solo lectura (sistema base, librerías y código).
- **Contenedor:** una imagen en ejecución. Sus datos se pierden al borrarlo si no hay volumen.
- **Registro (registry):** donde se suben y comparten imágenes (Docker Hub, `ghcr.io`).

Comandos básicos

Comando	Qué hace
`docker build -t mi-web:1.0 .`	Construye una imagen desde el Dockerfile de la carpeta actual
`docker images`	Lista las imágenes
`docker run -d --name mi-web -p 8080:80 mi-web:1.0`	Lanza un contenedor en segundo plano (`-d`), con nombre y puerto publicado
`docker run ... -v carpeta:/ruta ...`	Igual, montando una carpeta del equipo (volumen)
`docker ps`	Contenedores en marcha
`docker stop mi-web` / `docker rm mi-web`	Para / borra el contenedor
`docker login`, `docker tag`, `docker push`	Subir una imagen a un registro
`docker compose up -d --build`	Construye y levanta todos los servicios de `compose.yaml`
`docker compose ps`	Estado de los servicios
`docker compose down`	Borra contenedores y red, **conserva los volúmenes**
`docker compose down -v`	Borra también los volúmenes (se pierden los datos)

Instrucciones básicas del Dockerfile

- `FROM`: imagen base (siempre la primera; mejor con versión fija, p. ej. `nginx:stable-alpine`).
- `WORKDIR`: carpeta de trabajo dentro del contenedor.

- `COPY` : copia archivos del equipo a la imagen.
- `RUN` : ejecuta un comando **al construir** la imagen (p. ej. `npm install`).
- `CMD` : comando que se ejecuta **al arrancar** el contenedor.

```
FROM nginx:stable-alpine
COPY web/ /usr/share/nginx/html/
```

Apartado 11 · WSL (por encima)

- **Qué es:** Windows Subsystem for Linux. Permite usar un Linux completo dentro de Windows sin máquina virtual tradicional ni dual boot, con rendimiento casi nativo.
- **WSL 1:** traduce las llamadas de Linux al kernel de Windows; ligero, sin aplicaciones gráficas.
- **WSL 2:** usa una máquina virtual ligera con un **kernel Linux real**; rinde mejor, admite aplicaciones gráficas y es el que usa **Docker Desktop**.
- **Instalar:** en PowerShell como administrador, `wsl --install` (instala WSL 2 con Ubuntu); se comprueba con `wsl --list --verbose` .
- **Requisito clave:** tener la virtualización activada en la BIOS/UEFI.

12. Tus prácticas resueltas

Resumen de lo que entregaste en cada práctica; es el tipo de razonamiento que se puede pedir en el examen.

Tarea 1 – Elegir el sistema de archivos

Caso	Elección	Por qué	A qué se renuncia
Pendrive 128 GB, vídeos de 8 GB entre Windows, macOS y Linux	**exFAT**	FAT32 no admite archivos > 4 GB; NTFS en macOS solo se lee; ext4 no funciona en Windows ni macOS sin programas extra	Permisos y registro de cambios
Raíz de servidor Linux con varios usuarios	**ext4**	Guarda permisos por archivo y tiene journaling	—

Caso	Elección	Por qué	A qué se renuncia
Volumen donde se instala Windows 11	**NTFS**	Windows 11 solo se instala en NTFS; permisos, journaling, cifrado, archivos grandes	—
SD de cámara antigua que solo reconoce FAT32	**FAT32**	Un archivo no pasa de 4 GB: al llegar al límite la cámara lo corta (abre otro o deja de grabar). Se evita grabando en fragmentos y uniéndolos después, o bajando resolución/calidad	Vídeos > 4 GB en un solo archivo
Disco externo 2 TB para copias de Windows y Linux con permisos	**Dos particiones: NTFS + ext4**	NTFS guarda permisos de Windows pero no de Linux; ext4 al revés y Windows no lo lee. Alternativa: un solo NTFS (sin permisos Linux) o guardar las copias de Linux en un `.tar`, que conserva permisos	Un solo sistema que lo cumpla todo no existe
Partición EFI de un equipo UEFI	**FAT32**	La especificación UEFI exige FAT; es lo que el firmware sabe leer antes de cargar el SO	—

Tarea 2 – Dual boot

Caso A: portátil UEFI, SSD NVMe 512 GB, 16 GB RAM, Windows 11 (≥ 150 GB) + Ubuntu con /home separado y sin hibernación. **GPT** (Windows 11 en UEFI solo se instala sobre GPT; hasta 128 primarias; copia de la tabla al final).

N.º	Uso	Sist. archivos	Tamaño	Montaje / letra	Dispositivo
1	EFI (ESP)	FAT32	512 MB	/boot/efi	/dev/nvme0n1p1
2	Reservada Microsoft (MSR)	Sin formato	16 MB	—	/dev/nvme0n1p2
3	Windows 11	NTFS	160 GB	C:	/dev/nvme0n1p3
4	Recuperación de Windows	NTFS	1 GB	—	/dev/nvme0n1p4
5	Raíz Ubuntu	ext4	60 GB	/	/dev/nvme0n1p5

N.º	Uso	Sist. archivos	Tamaño	Montaje / letra	Dispositivo
6	Intercambio	swap	4 GB	[SWAP]	/dev/nvme0n1p6
7	Carpetas personales	ext4	resto (286,472 GB)	/home	/dev/nvme0n1p7

- ESP: una sola para los dos sistemas (Windows en `\EFI\Microsoft`, Ubuntu en `\EFI\ubuntu`); Microsoft pide mínimo 200 MB, tú diste 512 MB.
- swap 4 GB: para 16 GB de RAM sin hibernación; con hibernación harían falta unos 20 GB.
- /home separado: si hay que reinstalar Ubuntu no se pierden los datos.
- Nombres NVMe: `nvme0` controladora, `n1` namespace, `p1, p2…` particiones.
- 1 GB del fabricante = 10^9 bytes; Windows cuenta en GiB (1.073.741.824 bytes) → el disco aparece como ~476 GiB.

Orden: primero **Windows**, luego **Ubuntu** (Windows Boot Manager solo arranca Windows y se pone el primero; GRUB detecta Windows con `os-prober` y lo añade a su menú).

En la instalación de Ubuntu: instalación manual; indicar la ESP existente **sin formatearla** (si se formatea, se borra el arranque de Windows); gestor en `/dev/nvme0n1`. Hora desfasada: `timedatectl set-local-rtc 1`.

Si se instala primero Ubuntu y después Windows: Windows se pone el primero en el orden UEFI y entra directo, sin menú; Ubuntu sigue instalado (GRUB sigue en `\EFI\ubuntu`). Soluciones: poner "ubuntu" antes en el orden de arranque de la UEFI · `bcdedit /set {bootmgr} path \EFI\ubuntu\shimx64.efi` · Boot-Repair o reinstalar GRUB (`grub-install --target=x86_64-efi …` y `update-grub`) · `sudo update-grub` (o `GRUB_DISABLE_OS_PROBER=false`) para que detecte Windows. Con BIOS/MBR Windows sobrescribe el MBR y GRUB sí se borra. Windows no sabe redimensionar ext4 (hace falta GParted).

Caso B: BIOS clásica, disco 2 TB, 6 particiones, **MBR**: 3 primarias + 1 extendida con 3 lógicas.

Dispositivo	Tipo	Uso	FS	Tamaño	Montaje
/dev/sda1	Primaria (activa)	Sistema	ext4	100 GB	/
/dev/sda2	Primaria	Intercambio	swap	4 GB	[SWAP]
/dev/sda3	Primaria	Datos	ext4	1.296 GB	/datos
/dev/sda4	Extendida	Contenedor	—	600 GB	—
/dev/sda5-7	Lógicas	Pruebas 1-3	ext4	200 GB c/u	/pruebas1-3

MBR guarda inicio y tamaño en campos de 32 bits (sectores de 512 bytes): máximo 2^{32} × 512 B = 2 TiB; un disco "de 2 TB" (≈ 1,82 TiB) cabe entero. **¿GPT mejor?** Técnicamente sí (sin extendida/lógicas, copia de la tabla, sin límite de 2 TiB), pero Windows solo arranca desde GPT con UEFI; Linux sí, con una partición **BIOS boot** de 1 MB para GRUB. Con BIOS antigua o Windows previsto, MBR es lo seguro.

Tarea 3 – Ubuntu Desktop (en VMware, no VirtualBox)

Ubuntu 26.04.1 LTS "Resolute Raccoon". VM: 4 GB RAM, 2 núcleos, disco 25 GB VMDK sin reservar (equivale al VDI dinámico), red Bridged. **Equivalencias VirtualBox → VMware:** "Nueva" → File > New Virtual Machine · VDI → VMDK · Guest Additions → VMware Tools (`open-vm-tools`). Instalación: idioma, teclado, *Instalar Ubuntu*, selección predeterminada ("instalación normal"), programas privativos, **instalación manual** (45 % `/` , 50 % `/home` , 5 % swap), usuario, copia (~20 min). Particiones resultantes: `sda1` BIOS boot 1 MB (la añade el instalador: BIOS + GPT), `sda2` /home 12,50 GB, `sda3` / 11,13 GB, `sda4` swap 3,21 GB; total 26,84 GB porque 25 GiB = 26,84 GB. Post-instalación: `sudo apt update && sudo apt upgrade -y` (`&&` : la segunda solo si la primera va bien; `-y` responde sí), `sudo apt autoremove` para paquetes no necesarios, reiniciar con `sudo reboot` . Verificar con `lsb_release -a` , `lsblk` , `swapon --show` .

Tarea 4 – Ubuntu Server

Diferencias con Desktop: instalador **Subiquity** en modo texto (flechas, Tab, Espacio, Intro), sin opción "Probar Ubuntu", red configurada durante la instalación (IP fija), disco con **LVM** por defecto, ofrece servidor SSH, sin entorno gráfico (menos memoria y disco). VM: red **NAT** 192.168.111.0/24, IP fija 192.168.111.10 en `ens33` .

Disco "Use an entire disk" + LVM: `sda1` BIOS boot 1 MiB · `sda2` /boot ext4 2 GiB · `sda3` PV LVM 22,997 GiB → grupo `ubuntu-vg` → volumen lógico `ubuntu-lv` 11,496 GiB montado en `/` (ext4) + 11,50 GiB libres para ampliar con `lvextend` . Swap = archivo `/swap.img` en la raíz. LVM: partición → volumen físico (PV) → grupo de volúmenes → volúmenes lógicos (LV).

Incidencia: sin puerta de enlace ni DNS, `apt update` falla ("Fallo temporal al resolver archive.ubuntu.com") aunque `ip addr` muestre la IP. Solución con **netplan** (`/etc/netplan/50-cloud-init.yaml`): `addresses: [192.168.111.10/24]` , `routes: - to: default via: 192.168.111.2` , `nameservers: addresses: [192.168.111.2, 8.8.8.8]` ; aplicar con `sudo netplan apply` , comprobar con `ping -c 3 archive.ubuntu.com` . Diagnóstico: `ip addr` , `ip route` (sin línea `default` = falta la puerta de enlace). El nombre `ubuntu_server` quedó como `ubuntuserver` porque el guión

bajo no es válido en nombres de equipo (mejor `ubuntu-server`). Ubuntu Pro se omitió (`sudo pro attach` para activarlo después). Comandos de verificación: `lsblk`, `sudo vgs`, `sudo lvs`, `swapon --show`.

Tarea 5 – Docker

Parte 1 (nginx):

```
FROM nginx:stable-alpine
COPY web/ /usr/share/nginx/html/
```

`docker build -t mi-web:1.0 .` → `docker images` → `docker run -d --name mi-web -p 8080:80 mi-web:1.0` (nginx escucha en el 80 del contenedor, publicado en el 8080 del equipo). Con volumen (bind mount) para editar sin reconstruir: `docker run -d --name mi-web -p 8080:80 -v "${PWD}\web:/usr/share/nginx/html" mi-web:1.0`; comprobar con `docker inspect -f "{{json .Mounts}}" mi-web`. Opcional: subir a `ghcr.io` con `docker login ghcr.io`, `docker tag` y `docker push` (token con permiso `write:packages`).

Parte 2 (Node.js + MariaDB con Compose): contador de visitas en el puerto 3000; variables `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`.

```
FROM node:24-alpine
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "server.js"]
```

```
services:
  app:
    build: ./app
    ports:
      - "3000:3000"
    environment:
      DB_HOST: db
      DB_USER: usuario
      DB_PASSWORD: usuario123
      DB_NAME: visitas
    depends_on:
      - db
  db:
    image: mariadb:11.4
    environment:
      MARIADB_ROOT_PASSWORD: root123
      MARIADB_DATABASE: visitas
```

```
      MARIADB_USER: usuario
      MARIADB_PASSWORD: usuario123
    volumes:
      - datos-db:/var/lib/mysql
volumes:
  datos-db:
```

`docker compose up -d --build` y `docker compose ps`. Compose crea la red `parte2_default` y el volumen `parte2_datos-db`; `app` publica 3000, `db` solo expone 3306 dentro de la red (no publicado). `DB_HOST: db` funciona porque el nombre del servicio se resuelve dentro de la red de Compose.

Persistencia: `docker compose down` elimina contenedores y red **pero no los volúmenes** → el contador sigue (60). `docker compose down -v` borra también el volumen → base nueva y el contador vuelve a 1. En el temario se usa `docker-compose` con `version:`; hoy el comando es `docker compose` y `version:` ya no hace falta (así lo indica el enunciado).

Incidencias: (1) `failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine` → el cliente funcionaba pero no el motor; se reinició Docker Desktop (WSL 2 estaba correcto, `wsl --status`). (2) Fichero `DockerFile` con F mayúscula → error "failed to read dockerfile"; se renombró a `Dockerfile` (en dos pasos con `Rename-Item`).

13. Preguntas tipo examen (tapa la columna de respuesta y contesta)

Pregunta	Respuesta
Límite de archivo de FAT32, NTFS y ext4	4 GB, 16 EB, 16 TB
Pendrive de 128 GB con vídeos de 8 GB entre Windows, macOS y Linux	exFAT (FAT32 no admite > 4 GB; NTFS en macOS solo lectura; ext4 no funciona en Windows/macOS); se renuncia a permisos y registro de cambios
Sistema de archivos de la raíz de un servidor Linux con varios usuarios	ext4: permisos por archivo y journaling
Qué pasa en una SD con FAT32 al llegar a 4 GB y cómo evitarlo	La cámara corta el archivo (abre otro o deja de grabar); grabar en fragmentos y unirlos después, o bajar resolución/calidad

Pregunta	Respuesta
Disco externo de 2 TB para copias de Windows y Linux conservando permisos	Dos particiones: NTFS + ext4; alternativas: un solo NTFS (sin permisos Linux) o copias de Linux en `.tar`
Sistema de archivos de la partición EFI	FAT32 (lo exige UEFI; es lo que el firmware sabe leer)
Qué almacena la MFT	Los metadatos de todos los archivos de NTFS
MBR vs GPT	MBR: 512 bytes, hasta 2 TB, 4 primarias, BIOS. GPT: GUIDs, hasta 8 ZiB, 128 primarias, UEFI, copia de la tabla al final
Cuántas extendidas puede haber y cómo se numeran las lógicas	Una por disco (solo MBR); lógicas desde `/dev/sda5`
Qué es la MSR	Partición reservada de Microsoft en GPT, 16 MB, sin formato ni letra
Caso A: por qué GPT y qué particiones lleva	UEFI y Windows 11 solo en GPT; EFI 512 MB, MSR 16 MB, Windows 160 GB, recuperación 1 GB, / 60 GB, swap 4 GB, /home el resto
Por qué swap de 4 GB y /home separado	16 GB de RAM sin hibernación bastan con 4 GB (con hibernación ~20 GB); /home separado para no perder datos al reinstalar Ubuntu
Cómo se llama el disco NVMe en Linux	`/dev/nvme0n1`: `nvme0` controladora, `n1` namespace, `p1…` particiones
Por qué se instala primero Windows y después Ubuntu	El gestor de Windows no detecta Linux y se pone primero; GRUB sí detecta Windows y lo añade a su menú
Por qué indicar la ESP sin formatearla al instalar Ubuntu	Si se formatea, se borra el gestor de arranque de Windows
Qué pasa si se instala primero Ubuntu y luego Windows y cómo se arregla	Windows se pone primero en la UEFI y entra sin menú; poner ubuntu primero en la UEFI, `bcdedit /set {bootmgr} path \EFI\ubuntu\shimx64.efi`, Boot-Repair o `grub-install` + `update-grub`
Caso B: tabla MBR de 6 particiones	3 primarias (sda1-3) + 1 extendida (sda4) con 3 lógicas (sda5-7); MBR solo tiene 4 entradas

Pregunta	Respuesta
Límite de tamaño de MBR y si un disco de 2 TB cabe	$2^{32} \times 512$ B = 2 TiB; un disco de 2 TB (≈ 1,82 TiB) cabe entero
¿Sería GPT mejor en el caso B?	Sí para Linux (sin extendida/lógicas, copia de la tabla, sin límite); no si se quiere Windows (solo arranca GPT con UEFI) o la BIOS es muy antigua; GRUB con GPT en BIOS necesita partición BIOS boot de 1 MB
Por qué aparece `sda1` de 1 MB en tus VM	Partición BIOS boot que añade el instalador: BIOS + GPT
Particionado manual de Ubuntu Desktop y por qué	45 % `/`, 50 % `/home`, 5 % swap; /home separado para reinstalar sin perder datos
Equivalencias VirtualBox → VMware	Nueva → New Virtual Machine; VDI → VMDK; Guest Additions → VMware Tools (`open-vm-tools`)
Tipo de hipervisor de VirtualBox y VMware Workstation	Tipo 2 (aplicación sobre un SO anfitrión)
Modos de red de VirtualBox	NAT (por defecto, comparte IP del anfitrión), puente (IP propia en la red física), red interna (solo entre VM), solo-anfitrión (anfitrión + VM, sin salida)
`sudo apt update && sudo apt upgrade -y`	`update` descarga la lista de paquetes; `upgrade` instala las versiones nuevas; `&&` ejecuta la 2.ª solo si la 1.ª va bien; `-y` responde sí; `autoremove` borra paquetes ya no necesarios
Diferencias Ubuntu Server vs Desktop	Server: instalador Subiquity en texto, sin "Probar Ubuntu", red configurada al instalar (IP fija), LVM por defecto, SSH, sin entorno gráfico
Qué es LVM y cómo queda tu disco del Server	Partición → volumen físico (PV) → grupo `ubuntu-vg` → volumen lógico `ubuntu-lv` en `/`; `/boot` ext4 fuera de LVM; 11,5 GiB libres para `lvextend`
Dónde está la swap en Ubuntu Server	Archivo `/swap.img` dentro de la raíz

Pregunta	Respuesta
Por qué falló `apt update` en el Server y cómo se arregló	Faltaban puerta de enlace y DNS ("Fallo temporal al resolver"); se editó `/etc/netplan/50-cloud-init.yaml` (ruta por defecto `via 192.168.111.2`, nameservers) y `sudo netplan apply`
Comandos de diagnóstico de red	`ip addr` (interfaz e IP), `ip route` (sin línea `default` = falta puerta de enlace), `ping -c 3 archive.ubuntu.com`
Contenedor vs máquina virtual	Contenedor: comparte kernel del host, aislamiento medio, arranca en segundos; VM: Guest OS completo, aislamiento alto, arranca en minutos
Qué hace tu Dockerfile de la parte 1	`FROM nginx:stable-alpine` + `COPY web/ /usr/share/nginx/html/`
Qué hace `-p 8080:80`	Publica el puerto 80 del contenedor en el 8080 del equipo (equipo:contenedor)
Para qué sirve el volumen (bind mount) de la parte 1	Editar `index.html` y ver el cambio sin reconstruir la imagen ni reiniciar
Dockerfile de la parte 2	`FROM node:24-alpine`, `WORKDIR /app`, `COPY . .`, `RUN npm install`, `CMD ["node", "server.js"]`; RUN actúa al construir, CMD al iniciar
Qué define tu `compose.yaml`	Servicio `app` (build ./app, puerto 3000, variables `DB_*`, `depends_on: db`) y `db` (mariadb:11.4, variables `MARIADB_*`, volumen `datos-db:/var/lib/mysql`)
Por qué `DB_HOST: db` funciona y por qué 3306 no se publica	El nombre del servicio se resuelve dentro de la red de Compose; la base de datos solo se usa dentro de esa red
`docker compose down` vs `down -v`	`down` conserva el volumen (el contador sigue); `down -v` lo borra (el contador vuelve a 1)

14. Límites de este repaso

- **Fuentes:** solo las páginas del temario, tus prácticas y la lista de lo que entra que dio el profesor. No hay información externa.
- **Qué entra:** la lista del profesor. Fuera: el resto del apartado 1 (definición, clasificación, ejemplos de SO y tendencias), los modos y componentes del apartado 2, la instalación, aplicaciones y controladores del apartado 6, y entero el apartado 10 (buenas prácticas de Docker).
- **Lectura de las páginas:** se leyeron en una versión condensada, no literal; ante una duda concreta, mira la página original.
- **VMware frente a VirtualBox:** hiciste las prácticas con VMware; el temario habla de VirtualBox. Las equivalencias están en el apartado 12.